INCREMENTAL PLANAR NEAREST NEIGHBOURS WITH LOGARITHMIC QUERIES, NEAR-LOGARITHMIC INSERTIONS AND LINEAR SPACE

JOHN IACONO, YAKOV NEKRICH

Editorial scope. Only theor:semidyn2 is counted. Complete author arxiv2504.07366v1 native scientific text and all appendix proofs are retained under this version own CC BY4.0. Original arxiv=true and socg/eurocg=false state is preserved. The original bibliography command is supplied mechanically through the native author bbl at the end; no mathematical statements or proofs are rewritten. Basic and persistent/semi-online variants are not separate tasks.

Allowed standard prerequisites (editorial).

- Planar Voronoi diagrams, convex separators, point location and Bentley-Saxe logarithmic rebuilding. Author cited classical constructions and merge algorithms; full source-specific samples, pieces, jump-query and potential proofs retained.

1. Introduction

In the nearest neighbor problem a set of points S is stored in a data structure so that for a query point q the point $p \in S$ that is closest to q can be found efficiently. The nearest neighbor problem and its variants are among the most fundamental and extensively studied problems in computational geometry; we refer to e.g. [20] for a survey. In this paper we study dynamic data structures for the Euclidean nearest neighbor problem in two dimensions. We show that the optimal $O(\log n)$ query time for this problem can be achieved while allowing insertions in time $O(\log^{1+\epsilon})$.

Previous Work. See Table 1. In the static scenario the planar nearest neighbor problem can be solved in $O(\log n)$ time by point location in Voronoi diagrams. However the dynamic variant of this problem is significantly harder because Voronoi diagrams cannot be dynamized efficiently: it was shown by Allen et al. [2] that a sequence of insertions can lead to $\Omega(\sqrt{n})$ amortized combinatorial changes per insertion in the Voronoi diagram. A static nearest-neighbor data structure can be easily transformed into an insertion-only data structure using the logarithmic method of Bentley and Saxe [3] at the cost of increasing the query time to $O(\log^2 n)$. Several researchers [10, 12, 21] studied the dynamic nearest neighbor problem in the situation when the sequence of updates is random in some sense (e.g. the deletion of any element in the data structure is equally likely). However their results cannot be extended to the case when the complexity of a specific sequence of updates must be analyzed.

Using a lifting transformation [11], 2-d nearest neighbor queries can be reduced to extreme point queries on a 3-d convex hulls. Hence data structures for the dynamic convex hull in 3-d can be used to answer 2-d nearest neighbor queries. The first such data structure (without assumptions about the update sequence) was presented by Agarwal and Matoušek [1]. Their data structure supports queries in $O(\log n)$ time and updates in $O(n^{\varepsilon})$ time; another variant of their data structure supports queries in $O(n^{\varepsilon})$ time and updates in $O(\log n)$ time. A major improvement was achieved in a seminal paper by Chan [4]. The data structure in [4] supports queries in $O(\log^2 n)$ time, insertions in $O(\log^3 n)$ expected time and deletions in $O(\log^6 n)$ expected time. The update procedure can be made deterministic using the result of Chan and Tsakalidis [6]. The deletion time was further reduced to $O(\log^5 n)$ [16] and to $O(\log^4 n)$ [5]. This sequence of papers makes use of shallow cuttings, a general powerful technique, but, alas, all uses of it for the point location problem in 2-d have resulted in $O(\log^2 n)$ query times.

Even in the case of insertion-only scenario, the direct application of the 45-year-old classic technique of Bentley and Saxe [3] remains the best insertion-only method with polylogarithmic update before this work; no data structure with $o(\log^2 n)$ query time and polylogarithmic update time was described previously.

Date: arxiv:2504.07366; CC BY 4.0.

	Query	Insert	Delete
Bentley and Saxe 1980 [3]	$O(\log^2 n)$	$O(\log^2 n)$	Not supported
Agarwal and Matoušek 1995 [1]	$O(\log n)$	$O(n^\varepsilon)$	$O(n^\varepsilon)$
''	$O(n^\varepsilon)$	$O(\log n)$	$O(\log n)$
Chan 2010 [4]	$O(\log^2 n)$	$O(\log^3 n)$ †	$O(\log^6 n)$ †
Chan and Tsakalidis 2016 [6]	$O(\log^2 n)$	$O(\log^3 n)$	$O(\log^6 n)$
Kaplan et al. 2020 [16]	$O(\log^2 n)$	$O(\log^3 n)$	$O(\log^5 n)$
Chan 2020 [5]	$O(\log^2 n)$	$O(\log^3 n)$	$O(\log^4 n)$
Here	$O(\log n)$	$O(\log^{1+\varepsilon} n)$	Not supported

TABLE 1. Known results. Insertion and deletion times are amortized, † denotes in expectation.

Our Results. We demonstrate that optimal $O(\log n)$ query time and poly-logarithmic update time can be achieved in some dynamic settings. The following scenarios are considered in this paper:

(1) We describe a *semi-dynamic insertion-only* data structure that uses $O(n)$ space, supports insertions in $O(\log^{1+\varepsilon} n)$ amortized time and answers queries in $O(\log n)$ time.
(2) In the *semi-online* scenario, introduced by Dobkin and Suri [13], we know the deletion time of a point p when a point p is inserted, i.e., we know how long a point will remain in a data structure at its insertion time. We describe a semi-online fully-dynamic data structure that answers queries in $O(\log n)$ time and supports updates in $O(\log^{1+\varepsilon} n)$ amortized time. The same result is also valid in the *offline* scenario when the entire sequence of updates is known in advance.
(3) In the *offline partially persistent scenario*, the sequence of updates is known and every update creates a new version of the data structure. Queries can be asked to any version of the data structure. We describe an offline partially persistent data structure that uses $O(n \log^{1+\varepsilon} n)$ space, can be constructed in $O(n \log^{1+\varepsilon} n)$ time and answers queries in $O(\log n)$ time.

All three problems considered in this paper can be reduced to answering point location queries in (static) Voronoi diagrams of $O(\log n)$ different point sets. For example, we can obtain an insertion-only data structure by using the logarithmic method of Bentley and Saxe [3], which we now briefly describe. The input set S is partitioned into a logarithmic number of subsets $S_1, \ldots, S_f$ of exponentially increasing sizes. In order to find the nearest neighbor of some query point q we locate q in the Voronoi diagram of each set S_i and report the point closest to q among these nearest neighbors. Since each point location query takes $O(\log n)$ time, answering a logarithmic number of queries takes $O(\log^2 n)$ time.

The fractional cascading technique [8] applied to this problem in one dimension decreases the query cost to logarithmic by sampling elements of each S_i and storing copies of the sampled elements in other sets S_j, $j < i$. Unfortunately, it was shown by Chazelle and Liu [9] that fractional cascading does not work well for two-dimensional non-orthogonal problems, such as point location: in order to answer $O(\log n)$ point location queries in $O(\log n)$ time, we would need $\tilde{\Omega}(n^2)$ space, even in the static scenario.

To summarize, the two obvious approaches to the insertion-only problem are to maintain a single search structure and update it with each insertion, the second is to maintain a collection of static Voronoi diagrams of exponentially-increasing size and to execute nearest neighbor queries by finding the closest point in all structures, perhaps aided by some kind of fractional cascading. The first approach cannot obtain polylogarithmic insertion time due to the lower bound on the complexity change in Voronoi diagrams caused by insertions [2], and the second approach cannot obtain $O(\log n)$ search time due to Chazelle and Liu's lower bound [9]. Our main intellectual contribution is showing that the lower bound of Chazelle and Liu [9] can be circumvented for the case of point location in Voronoi diagrams. Specifically, a strict fractional cascading approach requires finding the closest point to a query point in each of the subsets S_i; we loosen this requirement: in each S_i, we either find the closest point or provide a certificate that the closest point in S_i is not the closest point in S. This new, powerful and more flexible form of fractional cascading is done by using a number of novel observations about the geometry of the problem. We imagine our general technique may be applicable to speeding up search in other dynamic search problems. Our method employs planar separators to sample point sets and uses properties of Voronoi diagrams to speed up queries. We explain our method and show how it can be applied to the insertion-only nearest neighbor problem in Section 2. A further modification of our method that improves the

insertion time and the space usage is described in Section B. We describe a partially persistent data structure in Section C. A semi-online data structure is described in Section D.

2. Basic Insertion-Only Structure

We present our basic insertion only structure in several parts. In the first part, the overview (§2.1), we present the structure where one needed function, jump, is presented as a black box. With the jump function abstracted, our structure is a combination of known techniques, notably the logarithmic method of Bentley and Saxe [3] and sampling. In §2.2 we present the implementation of the jump function and the needed geometric preliminaries. In contrast to combination of standard techniques presented in the overview which require little use of geometry, our implementation of the jump function is novel and requires thorough geometric arguments. We then fully describe the underlying data structures needed in § 2.3. Note that all arguments presented here are done with the goal of obtaining $O(\log n)$ queries and polylogarithmic insertion. We opt here for clarity of presentation versus reducing the number of logarithmic factors in the insertion, as the arguments are already complex.

2.1. **Overview.** All notation is summarized in Table 2 which can be found on the last page, page 15. We let S denote the set of points currently stored in the structure, and use n to denote $|S|$. In this section we set a constant d to 2. Even though for this section d is fixed, we will express everything as a function of d as the techniques used in Appendix B will use non-constant d.

Let $\mathcal{S} = \{S_1, S_2, \dots S_f\}$ denote a partition of S into sets of exponentially-increasing size where $f \coloneqq |\mathcal{S}| = \Theta(\log_d n)$ and $|S_i| = \Theta(d^i)$. Note that the partition of $\mathcal{S}$ into S_i is not unique. Let $NN(P, q)$ be the nearest neighbor of q in a point set P, which we assume to be unique. Given a point q, the computation of $NN(S, q)$ is the query that our data structure will support.

We now define a sequence of point sets $T_1, \dots T_f$. The intuition is that, as in classical fractional cascading [8], the set T_i contains all elements of S_i and a sample of elements from the sets T_j where $j > i$; this implies the last sets are equal: $T_f = S_f$. This sampling will be provided by the function $Sample_j(k)$ which returns a subset of T_j of size $O(|T_j|/d^{2k})$; while it will have other important properties, for now only the size matters.

We now can formally define T_i:

$$T_i \coloneqq S_i \cup \bigcup_{j=i+1}^{f} Sample_j(j-i)$$

From this definition we have several observations which we group into a lemma, the proof can be found in Appendix A:

Lemma 1. Facts about T_i

(1) $T_f = S_f$
(2) T_i *is a function of the* S_j*, for* $j \geq i$.
(3) $S = \cup_{i=1}^{f} T_i$
(4) $NN(S, q) \in \bigcup_{i=1}^{f} \{NN(T_i, q)\}$
(5) $|T_i| = \Theta(d^i)$
(6) For any i $\sum_{j=i+1}^{f} |Sample_j(j-i)| = \Theta(|T_i|)$

A note on notation. We assume the partition of S into the sets $\mathcal{S} = \{S_1, S_2, \dots, S_f\}$. Any further notation that includes a subscript, such as T_i, is a function of the S_j, for $j \geq i$. This compact notation makes explicit the dependence of $anything_i$ only on S_j, for $j \geq i$. This dependence in one direction only (i.e., on non-smaller indexes only) is crucial to the straightforward application of the standard Bentley and Saxe [3] rebuilding technique in the implementation of the data structure and the insertion operation described in section §2.3-2.4.

Voronoi and Delaunay. Let $Vor(P)$ be the Voronoi diagram of point set P, let $Cell(P, p)$ be the cell of a point p in $Vor(P)$, that is the locus of points in the plane whose closest element in P is p. Thus $q \in Cell(P, p)$ is equivalent to $NN(P, q) = p$. Let $|Cell(P, p)|$ be the complexity of the cell, that is, the number of edges on its boundary. Let $G(P)$ refer to the Delaunay graph of P, the dual graph of the Voronoi diagram of P; the degree of p in $G(P)$ is thus $|Cell(P, p)|$ and each point in P corresponds to a unique vertex in $G(P)$. Delaunay graphs are planar. To simplify the description, we will not distinguish between points in a planar set P and vertices of $G(P)$. For example, we will sometimes say that a point p has degree x or say that a point p' is a neighbor of p. We will find it useful to have a compact

notation for expressing the union of Voronoi cells; thus for a set of points $P' \subseteq P$, let $Cells(P, P')$ denote $\bigcup_{p \in P'} Cell(P, p)$.

Pieces and Fringes. Given a graph, $G = (V, E)$, and a set of vertices $V' \subseteq V$, the *fringe* of V' (with respect to G) is the subset of V' incident to edges whose other endpoint is in $V \setminus V'$. Let $G = (V, E)$ be a planar graph. For any r, Frederickson [14] showed the vertices of G can be decomposed[1] into $\Theta(|V|/r)$ *pieces*, so that: (i) Each vertex is in at least one piece. (ii) Each piece has at most r vertices in total and only $O(\sqrt{r})$ vertices on its fringe. (iii) If a vertex is a non-fringe vertex of a piece (with respect to G), then it is not in any other pieces. (iv) The total size of all pieces is in $\Theta(|V|)$. Intuitively, the pieces are almost a partition of V where those vertices on the fringe of each piece may appear in multiple pieces. Such a decomposition of G can be computed in time $O(|V|)$ [15, 19]. We will apply this decomposition to T_i, which is both a point set and the vertex set of $G(T_i)$, for exponentially increasing sizes of r.

Given integers $1 \leq k < j < f$, let

$$Pieces_j(k) \coloneqq \{Piece_j^1(k), Piece_j^2(k), \ldots Piece_j^{|Pieces_j(k)|}(k)\}$$

be a decomposition of T_j into $r = \Theta(|T_j|/d^{4k})$ subsets such that each subset $Piece_j^l(k)$ has size $O(d^{4k})$ and a fringe of size $O(d^{2k})$ with respect to $G(T_i)$. We let

$$Seps_j(k) \coloneqq \{Sep_j^1(k), Sep_j^2(k), \ldots Sep_j^{|Pieces_j(k)|}(k)\}$$

be defined so that $Sep_j^\ell(k)$ denotes the fringe of $Piece_j^\ell(k)$, and let $\overline{Sep}_j^\ell(k)$ be $Piece_j^\ell(k) \setminus Sep_j^\ell(k)$. Thus each $Piece_j^\ell(k)$ is partitioned into its fringe vertices, $Sep_j^\ell(k)$, and its interior non-fringe vertices $\overline{Sep}_j^\ell(k)$; note that $\overline{Sep}_j^\ell(k)$ may be empty if all elements of $Piece_j^\ell(k)$ are on the fringe.

Finally, we define $Sample_j(k)$ to be the union of all the fringe vertices:

$$Sample_j(k) \coloneqq \bigcup_{Sep \in Seps_j(k)} Sep$$

thus $Seps_j(k)$ is a partition decomposition of $Sample_j(k)$.

For any $k \in [1..j-1]$, the decomposition of T_j into $Pieces_j(k)$, the partition of each $Piece_j^\ell(k)$ into $Sep_j^\ell(k)$ and $\overline{Sep}_j^\ell(k)$, and the set $Seps_j(k)$ can all be computed in time $O(|T_j|)$ using [19] if the Delaunay triangulation is available; if not it can be computed in time $O(|T_j| \log |T_j|)$. Thus computing these for all valid i takes time and space $O(|T_j| \log |T_j| \log_d |T_j|)$ as $k < j = O(\log_d |T_j|)$.

One property of this sampling technique is that points in T_j with Voronoi cells in $Vor(T_j)$ of complexity at least k are included in T_i if $j > i$ and $j - i = O(\log_d k)$. By complexity of a region, we mean the number of edges that bound this region.

Lemma 2. *Given $i < j$, if $p \notin T_i$ and $p \in T_j$ then the complexity of $Cell(T_j, p)$ is $O(d^{4(j-i)})$.*

Proof. Suppose that $|Cell(T_j, p)| > cd^{4(j-i)}$, for a constant c chosen later, we will show this implies $p \in T_i$. Thus the degree of p in $G(T_j)$ is greater than $cd^{4(j-i)}$. Consider the piece $Piece_j^\ell(j-i)$ in $Pieces_j(j-i)$ that contains p. This piece has size at most $O(d^{4(j-i)})$, which is at most $cd^{4(j-i)}$ for some c (here we choose c). Thus in $G(T_j)$, p must have neighbors which are not in $Piece_j^\ell(j-i)$. By definition, p is thus in the fringe $Sep_j^\ell(j-i)$, which implies $p \in Sep_j^\ell(j-i)$. From the definition of T_i, $T_i \subset Sep_j^\ell(j-i)$ and which gives $p \in T_i$. □

While we cannot bound the complexity of any Voronoi cell in a fringe, we can bound the complexity of $\overline{Sep}_j^\ell(j-i)$, the cells inside a fringe. Intuitively, each piece has the fringe cells on its exterior and non-fringe cells on its interior; imagining the fringe cells of a piece as an annulus gives two boundaries, the exterior boundary of the fringes, which is the boundary between the cells of this and other pieces, and the interior boundary of the fringes, which is the boundary between the fringe cells and the interior cells in this piece. Crucially, while the exterior boundary could have high complexity, the interior boundary does not, which we now formalize:

Lemma 3. *The complexity of $Cells(T_j, \overline{Sep}_j^\ell(j-i))$ is $O(d^{4(j-i)})$.*

[1] We use the word *decomposed* to mean a division of a set into into a collection sets, the *decomposition*, whose union is the original set, but, unlike with a partition, elements may belong to multiple sets.

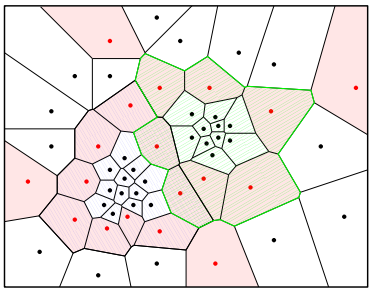


FIGURE 1. Part of a Voronoi diagram for a point set T_j. Two elements of $Pieces_j(k)$ have been highlighted, one in striped blue, call it $Piece_j^1(k)$, and one in striped green, call it $Piece_j^2(k)$. For each piece, the cells of fringe vertices are shaded red. Thus, the set $Sample_j(k)$ are the red verticies, and the region $Cells(T_j, Sample_j(k))$ is shaded red. The green-and-red shaded region is $Cells(T_j, Sep_j^2(k))$ and the green-but-not-red shaded region is $Cells(T_j, \overline{Sep}_j^2(k))$

.

Proof. See Figure 2. Each cell in $Cells(T_j, \overline{Sep}_j^\ell(j-i))$ is adjacent to either other cells in $Cells(T_j, \overline{Sep}_j^\ell(j-i))$ or cells in $Cells(T_j, Sep_j^\ell(j-i))$. The adjacency graph of these Voronoi regions is planar, and as

$\overline{Sep}_j^\ell(j-i) \cup Sep_j^\ell(j-i) = Piece_j^\ell(j-i)$, and recalling that $|Piece_j^\ell(j-i)| = O(d^{4(j-i)})$ gives the lemma. □

The Jump function: definition. At the core of our nearest neighbor algorithm is the function *Jump*, defined as follows. We will find it helpful to use $NN_R(q)$ for a range $R = [l, r]$ to denote $NN(\cup_{i\in[l,r]} T_i, q)$; for example $NN_{[1,k]}(q)$ is the nearest neighbor of q in $T_1, T_2, \ldots T_k$.

Intuitively, a call to $Jump(i, j, q, p_i, e_i)$ is used when trying to find the nearest neighbor of q, and assuming we know the nearest neighbor of q in $T_1, T_2 \ldots T_{(i+j)/2}$ seeks to provide information on whether there are any points that could be the nearest neighbor of q in $T_{(i+j)/2+1} \ldots T_j$. This information could be either a simple *no*, or it could provide the nearest neighbor of q for some prefix of these sets. Additionally, the edge of an the Voronoi cell of the currently known nearest neighbor in the direction of the query point is always passed and returned to aide the search using the combinatorial bounds from Lemma 8, point 4.

- **Input to** $Jump(i, j, q, p_i, e_i)$:
  - Integers i and j, where $j - i$ is required to be a power of 2. We use m to refer to $(j + i)/2$, the midpoint.
  - Query point q.
  - Point p_i where $p_i = NN(T_i, q)$.
  - The edge e_i on the boundary of $Cell(T_i, p_i)$ that the ray $\overrightarrow{p_i q}$ intersects.
- **Output**: Either one of two results, *Failure* or a triple $(j', p_{j'}, e_{j'})$
  - If *Failure*, this is a certificate that $NN_{(m,\min(j,f)]}(q) \neq NN_{[1,\min(j,f)]}(q)$
  - If a triple $(j', p_{j'}, e_{j'})$ is returned, it has the following properties:
    - The integer j' is in the range $(m, j]$ and $NN_{(m,j')}(q) \neq NN_{[1,j')}(q)$.
    - The point $p_{j'}$ is $NN(T_{j'}, q)$.
    - The edge $e_{j'}$ is on the boundary of $Cell(T_{j'}, p_j)$ that the ray $\overrightarrow{p_{j'} q}$ intersects.

We will show later that *Jump* runs in $O(j - i)$ time. Implementation details are deferred to Section 2.2.
The nearest neighbor procedure. A nearest neighbor query can be answered through a series of calls to the *Jump* function:

- Initialize $i = 1, j = 2$, p_1 to be $NN(T_1, q)$, and e_1 to be the edge of $Cell(T_1, p_1)$ crossed by the ray $\overrightarrow{p_1 q}$; all of these can be found in constant time as $|T_1| = \Theta(1)$. Initialize $p_{nearest}$ to p_1.
- Repeat the following while $\frac{i+j}{2} \leq f$:
  - Run $Jump(i, j, q, p_i, e_i)$. If the result is failure:
    - Set $j = j + (j - i)$
  - Else a triple $(j', p_{j'}, e_{j'})$ is returned:
    - If $d(p_{j'}, q) < d(p_{nearest}, q)$ set $p_{nearest} = p_{j'}$
    - Set $i = j'$ and set $j = j' + 1$
- Return $p_{nearest}$

We will show in the rest of this section that, given this jump function as a black box, we can correctly answer a nearest neighbor query in $O(\log n)$ time.
Correctness. The loop in the jump procedure will maintain the invariants that $p_{nearest}$ is $NN_{[1,\min(f,\frac{i+j}{2})]}(q)$, $i < j$ and $j - i$ is a power of two. The latter is because $j - i$ is set to either 1 or is doubled with each iteration of the loop. As to the first invariant, before the loop runs, the invariant requires $p_{nearest} = NN(T_1, q)$, which p_1 and $p_{nearest}$ are initialized to. During the loop we subscript i and j by new and old to indicate the value of variables at the beginning and at the end of the loop. Thus $p_{nearest} = NN_{[1,\min(f,\frac{i_{old}+j_{old}}{2})]}(q)$. Also if the loop runs, we know $\min(j_{old}, f) = j_{old}$. We distinguish between two cases depending on whether the jump function returns failure or a triple.

If the jump function returns failure, we know that $NN_{(\frac{i_{old}+j_{old}}{2},\min(j_{old},f)]}(q) \neq NN_{[1,\min(j_{old},f)]}(q)$. Using this fact we can go from the invariant in terms of the old variables to the new ones:

$$
\begin{aligned}
p_{nearest} &= NN_{[1,\min(f,\frac{i_{old}+j_{old}}{2})]}(q) && \text{Invariant}\\
&= NN_{[1,\frac{i_{old}+j_{old}}{2}]}(q) && \frac{i_{old}+j_{old}}{2} \le f\\
&= NN_{[1,\min(j_{old},f)]}(q) && NN_{(\frac{i_{old}+j_{old}}{2},\min(j_{old},f)]}(q) \neq NN_{[1,\min(j_{old},f)]}(q)\\
&= NN_{[1,\min(f,\frac{i_{old}+j_{old}+(j_{old}-i_{old})}{2})]}(q) && \text{math}\\
&= NN_{[1,\min(f,\frac{i_{new}+j_{new}}{2})]}(q) && i_{new} = i_{old}; j_{new} = j_{old} + (j_{old} - i_{old})
\end{aligned}
$$

Now we consider the case when a triple $(j', p_{j'}, e_{j'})$ is returned by the *Jump* function. We know that $NN_{(m,j']}(q) \neq NN_{[1,j']}(q)$, and using the same logic as from the failure case we can conclude that the old $p_{nearest}$ is $NN_{[1,j')}(q)$. The code sets $p_{nearest}$ to the point that is closer to q among the old $p_{nearest}$, equal to $NN_{[1,j')}(q)$, and $p_{j'}$, which is $NN(T_{j'}, q)$. Thus the new $p_{nearest}$ is equal to $NN_{[1,j']}(q)$ (note the closed interval) which we can rewrite to get the invariant as follows, using the fact that the subscript of NN is only dependent on the integers in the given range:

$$NN_{[1,j']}(q) = NN_{[1,i_{new}]}(q) = NN_{[1,\frac{i_{new}+i_{new}+1}{2}]}(q) = NN_{[1,\frac{i_{new}+j_{new}}{2}]}(q)$$

When the loop finishes, we have $j > f$ and thus

$$p_{nearest} = NN_{[1,\min(f,\frac{i+j}{2})]}(q) = NN_{[1,f]}(q) = \underset{p\in\{T_k | k\in[1,f] \text{ and } k\in\mathbb{Z}\}}{\arg\min} d(p,q) = NN(S,q)$$

Running time.

Lemma 4. *Given the Jump function, the running time of the nearest neighbor search function in* $O(\log n)$.

Proof. We use a potential argument. Let i_t and j_t denote the values of i and j after the loop has run t times, let a_t denote the runtime of the tth iteration of the loop, and let T denote the number of times the loop runs. The total runtime is thus $O(1) + \sum_{t=1}^{T} a_t$. Define $\Phi_t \coloneqq c(2i_t + j_t)$, for some constant c chosen so that $a_t \le \frac{c}{2}(j_j - i_t)$; as $i_0 = 1$ and $j_0 = 2$, $\Phi_0 = 4c$. (recall that the runtime of Jump is $O(j - i)$ for constant d).

We will argue that for all t, $a_t \le \Phi_t - \Phi_{t-1} + O(1)$. Summing, this gives $\sum_{t=1}^{T} a_t \le \Phi_T - \Phi_0 + O(T)$. Since i, j and T are in the range $[1, f]$, this bounds $\sum_{t=1}^{T} a_t$ by $3cf + O(f) = \Theta(\log n)$. Thus all that remains is to argue that $a_t \le \Phi_t - \Phi_{t-1}$ for the two cases where the tth run of Jump ends in failure or returns a triple.

Case 1, Jump returns failure. In the case of failure, i remains the same, thus $i_t = i_{t-1}$ but j increases by $j - i$, thus $j_t = 2j_{t-1} - i_{t-1}$. Thus the potential increases by $c(j - i)$.

$$
\begin{aligned}
\Phi_t - \Phi_{t-1} &= c(2i_t + j_t) - c(2i_{t-1} + j_{t-1})\\
&= c(2i_{t-1} + 2j_{t-1} - i_{t-1}) - c(2i_{t-1} + j_{t-1})\\
&= c(j_{t-1} - i_{t-1})\\
&\ge a_t
\end{aligned}
$$

Case 2, Jump returns a triple. i_t is set to j' and j_t changes to $j' + 1$. The key observation is that, due to the invariant in our correctness argument, $j_t \ge \frac{i_{t-1}+j_{t-1}}{2}$. Thus $i_t = j' \ge j_{t-1} \ge \frac{i_{t-1}+j_{t-1}}{2}$ and $j_t = j' + 1 \ge j_{t-1} + 1 \ge \frac{i_{t-1}+j_{t-1}}{2} + 1$ and the potential change is

$$
\begin{aligned}
\Phi_t - \Phi_{t-1} &= c(2i_t + j_t) - c(2i_{t-1} + j_{t-1})\\
&\ge c(2 \cdot \frac{i_{t-1} + j_{t-1}}{2} + \frac{i_{t-1} + j_{t-1}}{2} + 1) - c(2i_{t-1} + j_{t-1})\\
&= \frac{c}{2}(j_{t-1} - i_{t-i}) + c\\
&\ge a_t
\end{aligned}
$$

□

2.2. **The jump function.**

2.2.1. *Basic geometric facts.* We begin with our most crucial geometric lemma, the one that we build upon to make our algorithm work. Informally, given point sets A and B which possibly have elements in common, and a query point q, if the closest point to q in B is also in A, then the closest point to q in $A \cup B$ is in A, and not in $B \setminus A$.

Lemma 5. *Given i, j, $i < j$, suppose that there is a point q such that $q \in Cell(T_i, p_i) \cap Cell(T_j, p_j)$ for some $p_j \in Sample_j(j-i)$. Then $q \in Cell(T_i \cup T_j, p_i)$, or equivalently $NN(T_i \cup T_j, q) = p_i$.*

Proof. Since $p_j \in Sample_j(j-i)$, $p_j \in T_i$ by the definition of T_i. Since $q \in Cell(T_i, p_i)$, $dist(q, p_i) \leq dist(q, p_j)$. However, $q \in Cell(T_j, p_j)$ and p_j is the closest point to q in T_j: $dist(q, p_j) \leq dist(q, p'_j)$ for all p'_j in T_j. Combining $dist(q, p_i) \leq dist(q, p_j)$ with $dist(q, p_j) \leq dist(q, p'_j)$ for all p'_j in T_j gives the lemma. □

Lemma 6. *If all elements of a point set P are in $Cell(S, p)$ for some point set S and $p \in S$, then all elements of the convex hull of P are in $Cell(S, p)$ as well.*

Proof. Immediate as $Cell(S, p)$ is convex. □

Lemmata 5 and 6 give us a tool to determine which parts of the Voronoi cell of some p_i in $\mathrm{Vor}(T_i)$ must also be part of the Voronoi cell of p_i in $\mathrm{Vor}(T_i \cup T_j)$. We define this region as $Hull_i(j, p_i)$, and then prove its properties.

Let $Hull_i(j, p_i)$, for some $p_i \in T_i$, denote the convex hull of

$$\{p_i\} \cup (Cell(T_i, p_i) \cap Cells(T_j, Sample_j(j-i))).$$

See Figure 4 for an example.

Lemma 7. *$Hull_i(j, p_i) \subseteq Cell(T_i \cup T_j, p_i)$ and thus if $q \in Hull_i(j, p_i)$, $NN(q, S) \notin T_j \setminus T_i$*

Proof. The point p_i is in its own cell, $Cell(T_i \cup T_j, p_i)$, and by Lemma 5, all elements of $Cell(T_i, p_i) \cap Cells(T_j, Sample_j(j-i))$ are also in $Cell(T_i \cup T_j, p_i)$. Thus the convex hull of these points is a subset of $Cell(T_i \cup T_j, p_i)$ by Lemma 6. □

Thus, if a query point q is in $Hull_i(j, p_i)$, then, by Lemma 7, the set T_j can be ignored because $dist(q, p_i) \leq dist(q, p')$ for any $p' \in T_j \setminus T_i$.

Geometrically, determining whether a point in $Cell(T_i, p_i)$ is a $Hull_i(j, p_i)$ and thus a full search in $\mathrm{Vor}(T_j)$ can be skipped is what our jump function does. We now need to turn to examining the combinatorial issues surrounding $Hull_i(j, p_i)$ and its interaction with $Cell(T_i, p_i)$ as we need the complexity of the regions examined to be bounded in such a way to allow efficient searching to see if a point is in $Cell(T_i, p_i)$. We begin by defining the part of $Cell(T_i, p_i)$ that is not in $Hull_i(j, p_i)$ as $\overline{Hull}_i(j, p_i)$ and proving a number of properties of this possibly disconnected region.

Lemma 8. *Consider the region*

$$\overline{Hull}_i(j, p_i) \coloneqq Cell(T_i, p_i) \setminus Hull_i(j, p_i)$$

(1) Each connected component of $\overline{Hull}_i(j, p_i)$ is a subset of the union of Voronoi cells in one element of $Pieces_j(j-i)$; that is, each connected component of $\overline{Hull}_i(j, p_i)$ is a subset of $Cells(T_j, Piece_j^\ell(j-i))$ for some $Piece_j^\ell(j-i) \in Pieces_j(j-i)$.

(2) $\overline{Hull}_i(j, p_i)$ intersects each bounding edge of $Cell(T_i, p_i)$ in at most two connected components, each of which includes a vertex of $Cell(T_i, p_i)$.

(3) Any line segment $\overline{p_i q}$, where q is on the boundary of $Cell(S_i, p_i)$ intersects $\overline{Hull}_i(j, p_i)$ in at most one connected component that, if it exists, includes q.

(4) Let $\overline{qr}$ be a boundary edge of $Cell(T_i, p_i)$. The solid triangle $\triangle p_i q r$ intersects at most $d^{O(j-i)}$ edges on the boundary separating $Hull_i(j, p_i)$ from $\overline{Hull}_i(j, p_i)$.

Proof. (1) Suppose one connected component $\overline{Hull}_i(j, p_i)$ contains points p, p' in both $Cells(T_j, Piece_j^\ell(j-i))$ and $Cells(T_j, Piece_j^{\ell'}(j-i))$ where $Piece_j^\ell(j-i)$ and $Piece_j^{\ell'}(j-i)$ are different elements of $Pieces(S_j, j-i)$. Consider a polyline that connects p and p' while remaining in the same connected component of $\overline{Hull}_i(j, p_i)$. Such a polyline must cross, at some point, some cell $Cell(T_j, p_j)$ of some $p_j \in T_j$ where $p_j \in Piece_j^\ell(j-i)$ but where the cell $Cell(T_j, p_j)$ is adjacent to at least one other cell in $Vor(T_j)$ that is not in $Piece_j^\ell(j-i)$. Thus p_j is by definition in $Sep_j^\ell(j-i)$; thus p_j is in $Hull_i(j, p_i)$ by its definition in Lemma 7. But this contradicts p_j is in $\overline{Hull}_i(j, p_i)$.

(2) If this does not hold, there are points q_2 and q_3 on $\overline{Hull}_i(j, p_i)$, with q_2 closer to j than q_3, such that $q_2 \notin \overline{Hull}_i(j, p_i)$ and $q_3 \in \overline{Hull}_i(j, p_i)$. But this cannot happen: We know $p \in Hull_i(j, p_i)$ by construction and if p and q_2 are in $Hull_i(j, p_i)$, q_2 must be as well because the hull is a convex set.

(3) By a similar argument as the last point, if this did not hold, there would be points p_1, q_1, q_2, q_3 in order on $\overline{p_i q}$, where there are some points q_1, q_2, q_3, q_4, q_5, in order, such that $q_1, q_3, q_5 \in \overline{Hull}_i(j, p_i)$ and $q_2, q_4 \in Hull_i(j, p_i)$. But this cannot happen: if q_2 and q_4 are in $Hull_i(j, p_i)$, q_3 must be as well because the hull is a convex set.

(4) The complexity of $Hull_i(j, p_i) \cap \triangle p_i q r$ is at most the complexity of $Cells(T_j, \overline{Seps}_j^{\ell}(j-i))$ and $Cells(T_j, \overline{Seps}_j^{\ell'}(j-i))$, where $q \in Cells(T_j, \overline{Seps}_j^{\ell}(j-i))$ and $r \in Cells(T_j, \overline{Seps}_j^{\ell'}(j-i))$. By Lemma 3, the complexity of these regions (within the triangle $\triangle p_i q r$) is at most $d^{O(j-i)}$. Taking the convex hull only decreases the complexity of the objects on which a hull is defined.

□

We can use these geometric facts to present the following corollary, which shows how we can use inclusion in $Hull_i(j, p_i)$ to determine where to find the nearest neighbor of q in T_j or to determine that $NN(S, q)$, q's nearest neighbor in S, is not in T_j *without needing to find the nearest neighbor of q in T_j*. We emphasize this is the main novel idea, since, as discussed in the introduction, finding the nearest neighbors in every T_j in logarithmic time is not possible.

Corollary 1. *Given:*

- *i and j, $i < j$*
- *a query point q*
- *a point p_i where $p_i = NN(T_i, q)$*
- *the edge $\overline{v_1 v_2}$ on the boundary of $Cell(T_i, p_i)$ that the ray $\overrightarrow{p_i q}$ intersects.*

then, by testing q against the part of $Hull_i(j, p_i)$ that is inside $\triangle p_i v_1 v_2$ and has complexity $d^{O(j-i)}$ one of the following is true:

- *If q is inside $Hull_i(j, p_i)$, then q is in $Cell(T_i \cup T_j, p_i)$ and $NN(S, q)$ is not in T_j.*
- *If q is outside $Hull_i(j, p_i)$ (and thus inside $\overline{Hull}_i(j, p_i)$): then $NN(T_j, q)$ is in the same element of $\overline{Seps}_j(j-i)$ as either v_1 or v_2.*

2.2.2. *Implementing the jump function.* We now use one additional idea to speed up the jump function: While testing if q is inside or outside of the part of $Hull_i(j, p_i)$ that intersects $\triangle p_i v_1 v_2$ can be done in time $O((j-i)\log d)$ (since the complexity of this part of the hull is $d^{O(j-i)}$ by Lemma 8, point 4), we can in fact do something stronger. We can test if q is inside or outside of the part of $Hull_i(j', p_i)$ that intersects $\triangle p_i v_1 v_2$, **for all** $\frac{i+j}{2} < j' < j$, in the same time $O((j-i)\log d)$. This is because the complexity of the subdivision of the plane induced by $\frac{j-i}{2}$ hulls of size $d^{O(j-i)}$ has complexity $d^{O(j-i)} \cdot O((j-i)^2)$ and thus we can determine in time $O((j-i)\log d)$ which is the smallest j' in the range $i < j' \leq j$ where q is not inside $Hull_i(j', p_i)$, or determine that for all j' in the range $i < j' \leq j$.

Thus we obtain the final jump procedure:

Method to compute $Jump(i, j, q, p_i, e_i)$**:**

(1) Test to see what is the smallest j', $\frac{i+j}{2} < j' \leq j$ such that q is outside of the part of $Hull_i(j', p_i)$ that intersects $\triangle p_i v_1 v_2$. This will be done in time $O(\log d(j-i))$ with the convex hull search structure described in Section 2.3. If it is inside all such hulls, then $NN(S, q) \notin T_{j'}$ for all j' such that $\frac{i+j}{2} < j' \leq j$ and failure is returned. Otherwise:

(2) Let $Piece_{v_1}$ and $Piece_{v_2}$ denote the elements of $Pieces_j(j'-i)$ that contain v_1 and v_2. These will be precomputed and accessible in constant time with the piece lookup structure described in Section 2.3.

(3) Search in $Vor(T_{j'}, Piece_{v_1})$ and $Vor(T_{j'}, Piece_{v_2})$ to find $NN(Piece_{v_1} \cup Piece_{v_2}, q)$, call it $p_{j'}$. Note that, $p_{j'}$ is $NN(T_j, q)$. As both $Vor(T_{j'}, Piece_{v_1})$ and $Vor(T_{j'}, Piece_{v_2})$ have complexity $O(d^{j'-i})$, this can be done in time $O((j-i)\log d)$ with the piece interior search structure described in Section 2.3.

(4) Find the edge $e_{j'}$ bounding $Cell(T_{j'}, p_{j'})$ that the ray $\overrightarrow{p_{j'} q}$ intersects. As $Cell(T_{j'}, p_{j'})$ has complexity $d^{O(j'-i)}$, this can be done in time $O((j-i)\log d)$ with binary search.

2.3. **Data structures needed.** The data structure is split into levels, where level i consists of:

I. S_i,

II. T_i,
III. The Voronoi diagram of T_i, $Vor(T_i)$, and a point location search structure for the cells of $Vor(T_i)$,
IV. The Delaunay triangulation of T_i, $G(T)$.
V. Additionally, we keep for each j, $1 \leq j < i$:
   i. The partition of T_i into $Pieces_j(k) \coloneqq \{Piece_j^1(k), Piece_j^2(k), \ldots Piece_j^{|Pieces_j(k)|}(k)\}$
   ii. The partition of each $Piece_j^\ell(k)$ into $Sep_j^\ell(k)$ and $\overline{Sep}_j^\ell(k)$
   iii. The set $Sample_j(k) \coloneqq \bigcup_{Sep \in Seps_j(k)} Sep$

For any level i, this information can be computed from T_i in time $O(|T_i| \log |T_i|)$ using [19][Theorem 3] to compute the partition into pieces, and standard results on Delaunay/Voronoi construction.

Additionally, less elementary data structures are needed for each level, which we describe separately: the convex hull search structure, the piece lookup structure, and the piece interior search structure.

Convex hull search structure. For level i, a convex hull structure is built for every combination of:

- A point p_i in T_i
- An edge e_i of $Cell(T_i, p_i)$
- An index j where $i < j$, $\frac{i+j}{2} \leq f$ and $j - i$ is a power of 2.

A convex hull structure answers queries of the form: given a point q in $Cell(T_i, p_i)$, return the smallest j', $\frac{i+j}{2} < j' \leq j$ such that q is outside of the part of $Hull_i(j', p_i)$ that intersects $\triangle p_i v_1 v_2$, where v_1 and v_2 are endpoints of e_i. There are $O(|T_i| \log \log_d |T_i|)$ such structures as j is at most $f = \Theta(\log_d n)$, and the complexity of a Voronoi diagram $Vor(T_i)$ is linear in the number of points it is defined on.

The method used is to simply store a point location structure which contains subdivision within $\triangle p_i v_1 v_2$ formed by the overlay of all boundaries of $Hull_i(j', p_i)$, for $\frac{i+j}{2} < j' \leq j$. As previously mentioned the complexity of this overlay is $O(d^{j-i} \cdot (j-i)^2)$ and thus point location can be done in the logarithm of this, which is $O((j-i) \log d)$.

As noted earlier there are $O(|T_i| \log \log_d n)$ structures, each of which takes space at most $O(j - i) = O(\log_d n)$ plus the number of intersections in the point location structure within the triangle. The $O(j-i)$ comes from the at most 2 edges from each hull that can pass through the triangle without intersection. For a given j, the sum of the complexities of $Hull_i(j, p_i)$ over all $p_i \in T_i$ is $O(T_i)$. As each hull edge can intersect at most $O(j - i) = O(\log_d n)$ other hull edges, that bounds the total space needed over one j to be $O(n \log_d n)$. The overall space usage is $O(n \log_d^2 n)$.

Piece lookup structure. Level i of the piece lookup structure contains for j, $i < j \leq \min(2i, f)$ and for each vertex v_i of $Vor(T_i)$ the index of which piece $Piece_j^\ell(j-i) \in Pieces_j^\ell(j-i)$ has q in $Cells(T_j, Piece_j^\ell(j-i))$. This can be precomputed using the point location search structure for $Vor(T_j)$ in time $O(\log d^j) = O(j \log d) = O(i) = O(\log n)$ for fixed d for each of the $O(|T_i|)$ vertices of $Vor(T_i)$. Summing over the choices for j gives a total runtime of $O(|T_i| \log^2 n)$ to pre-compute all answers. The space usage is $O(|T_i| \log n)$.

Piece interior search structure. For each $1 \leq i < j \leq f$ we store a point location structure that supports point location in time $O(j - i)$ in the Voronoi diagram for each set of points in $Pieces_j(j - i)$. Any standard linear-sized point location structure, such as that of Kirkpatrick [18], suffices since each element of $Pieces_j(j - i)$ has $O(4^{j-i})$ elements. For any fixed j, there are $j - 1$ choices for i, and the sets in $Pieces_j(j - i)$ partition T_i. Thus the total size of all these structures is $O(|T_j| \log n)$. The construction cost, given the $Pieces_j(j-i)$, incurs another logarithmic factor due to the need to construct the Voronoi diagram and the point location structure (we do not assume each piece is connected). Thus the piece interior search structure for level j is constructed in $O(|T_j| \log^2 n)$ time.

2.4. **Insertion time.** Our description of the data structures needed can be summarized as follows:

Lemma 9. *Level i of the data structure can be built in time and space $O(|T_i| \log^2 n)$ given all levels $j > i$.*

Insertion is thus handled by the classic logarithmic method of Bentley and Saxe [3] which transforms a static construction into a dynamic structure, and which we briefly summarize. To insert, we put the new point into S_1 and rebuild level 1. Every time a set S_i exceeds the upper limit of $\Theta(d^i)$, half of the items are moved from S_i to S_{i+1} and all levels from $i + 1$ down to S_1 are rebuilt. So long as the upper and lower constants in the big Theta are at least a constant factor apart, the amortized insertion cost is $O(\log n)$ times the cost per item to rebuild a level, thus obtaining:

Lemma 10. *Insertion can be performed with an amortized running time of $O(\log^3 n)$.*

The performance of our data structure can be summarized as follows:

Theorem 1. *There exists a semi-dynamic insertion-only data structure that answers two-dimensional nearest neighbor queries in $O(\log n)$ time and supports insertions in $O(\log^3 n)$ amortized time. The data structure uses $O(n \log^2 n)$ space.*

Appendix A. Proof of Lemma 1

Lemma 1. Facts about T_i

(1) $T_f = S_f$
(2) T_i *is a function of the* S_j*, for* $j \geq i$.
(3) $S = \cup_{i=1}^{f} T_i$
(4) $NN(S, q) \in \bigcup_{i=1}^{f} \{NN(T_i, q)\}$
(5) $|T_i| = \Theta(d^i)$
(6) For any i

$$\sum_{j=i+1}^{f} |Sample_j(j-i)| = \Theta(|T_i|)$$

Proof.

(1) $T_f \coloneqq S_f \cup \bigcup_{j=f+1}^{f} Sample_j(j-i) = S_f$
(2) Follows from the definition of T_i and that $Sample_j(j-i)$ is a function of T_j and $j-i$ and the fact that $j > i$.
(3) Follows from the fact that the S_i are a partition of S and all T_i are subsets of S.
(4) Follows immediately from the previous point.
(5) Since $|S_i| = \Theta(d^i)$ and $|T_f| = |S_f| = \Theta(d^f)$, this can be verified by solving this recurrence:

$$|T_i| \leq cd^i + \sum_{j=i+1}^{f} \frac{|T_j|}{d^{2(j-i)}}$$

(6)

$$\sum_{j=i+1}^{f} |Sample_j(j-i)| = \Theta\left(\sum_{j=i+1}^{f} \frac{|T_j|}{d^{2(j-i)}}\right) = \Theta\left(|T_i| \sum_{j'=1}^{\infty} \frac{1}{2^{j'}}\right) = \Theta(|T_i|)$$

□

Appendix B. Faster Updates

The data structure described in Theorem 1 achieves optimal query time but uses superlinear space. Superlinear space usage is needed to implement the Jump procedure: our implementation stores a polylogarithmic number of data items for each Voronoi edge in T_i. In this section we explain how the data structure can be modified so that both $O(n)$ space usage and more efficient update time are achieved. Our improvement is based on the following idea: We maintain the Voronoi diagram for an auxiliary subset $T_i' \subset T_i$ that contains $|T_i|/\operatorname{polylog}(n)$ points. Before we start the Jump procedure, we locate q in $Vor(T_i')$ and use a Voronoi cell of $Vor(T_i')$ as the starting point of the Jump procedure. A detailed description of the modified data structure is provided below.

Data Structure. We change the value of the parameter d and set $d = \log^{\varepsilon} n$ for a constant $\varepsilon > 0$. We re-define T_i so that T_i contains at most $\frac{|T_j|}{\log^3 n}$ points from every set T_j, $j > i$: let $g = \log^6 n$ points, let $h = \text{pow2}((1/4)\log_d g)$, and let

$$T_i \coloneqq S_i \cup \bigcup_{j=i+1}^{f} Sample_j(\max(\text{pow2}(j-i), h))$$

where $\text{pow2}(x) = 2^{\lceil \log_2 x \rceil}$. Thus each T_j is divided into pieces so that each piece contains $\Omega(\max(d^{4(j-i)}, g))$ points and the total number of pieces is $O(\min(\frac{|T_j|}{d^{4(j-i)}}, \frac{|T_j|}{g}))$. The number of points that are copied from T_j, $j > i$, to T_i does not exceed $O(\frac{|T_j|}{\sqrt{g}})$. Hence the set $T_i \setminus S_i$ contains $O(\frac{|T_i| \log n}{\sqrt{g}}) = O(\frac{T_i}{\log^2 n})$ points. We define $T_i' = Sample_h(T_i) \cup (T_i \setminus S_i)$. We construct convex hulls $Hull_i(j, p)$ for every set T_i', for every

$j > i$, and for all $p \in T'_i$. We also construct all auxiliary data structures for convex hull search described in Section 2.3 for every set T'_i. Since T'_i contains $O(\frac{|T_i|}{\log^2 n})$ points, all data structures can be constructed in $O(|T_i|)$ time and use $O(|T_i|)$ space.

Lemma 11. *If $p_i = NN(T_i, q)$ is known, we can find $p'_i = NN(T'_i, q)$ in $O(\log\log n)$ time.*

Proof. For every point $p \in T_i$ we store the index l such that p_i is in $Piece^l_i(h)$. For every set $P^l = T'_i \cap Piece^l_i(h)$, we store its Voronoi diagram and a point location data structure. The set $T'_i \cap Piece^l_i(h)$ contains polylog(n) points because $Piece^l_i(h)$ contains polylog(n) points. Hence we can find $p^l = NN(T'_i \cap Piece^l_i(h), q)$ in $O(\log\log n)$ time.

Suppose that $p'_i \neq p^l$. Then q is in $Cell(T'_i, p')$ such that the point p' is not in $Piece^l_i(h)$. The segment $\overline{qp'}$ must intersect some $Cell(T_i, p_s)$ such that $p_s \in Sep^l_i(h)$. Consider an arbitrary planar point q' on $\overline{qp'}$ such that q' is in $Cell(T_i, p_s)$. Since q' is in $Cell(T_i, p_s)$, $dist(p_s, q') < dist(p', q')$. On the other hand, q' in $Cell(T'_i, p')$ by convexity of Voronoi cells. Hence $dist(p', q') < dist(p_s, q')$. A contradiction. Hence $p'_i = p^l$ and we can find p'_i in $O(\log\log n)$ time. □

Lemma 12. *Let $p_i = NN(T_i, q)$ and $p'_i = NN(T'_i, q)$. If the edge v_i of $Cell(T_i, p_i)$ intersected by $\overrightarrow{p_i q}$ is known, we can find the edge v'_i of $Cell(T'_i, p'_i)$ intersected by $\overrightarrow{p'_i q}$ in $O(\log\log n)$ time.*

Proof. We already explained in Lemma 11 how $p'_i = NN(T'_i, q)$ can be found in $O(\log\log n)$ time. We distinguish between two cases.

1. $p'_i \neq p_i$. We consider the ray $\overrightarrow{p'_i q}$. Let q_e denote the point where $\overrightarrow{p'_i q}$ intersects a cell $Cell(p'_e, q)$ for some $p'_e \in T'_i$. If p_i is in $Piece^l_i(h)$, then every point r on $\overline{p'_i q_e}$ is in $Cell(T_j, p)$ for some $p \in Piece^l_i(h)$. Using the same arguments as in Lemma 11, the point r is in $Cell(T'_i, p_l)$ for some $p_l \in Piece^l_i(h) \cap T'_i$. Since the point q_e is in $Cell(T_i, p_e)$, q_e is in $Cell(T'_i, p_e)$. Hence $\overrightarrow{p'_i q}$ intersects an edge v'_i at some point r, such that r is on $\overline{p'_i q_e}$. Therefore the edge v'_i separates $Cell(T'_i, p'_i)$ and $Cell(T'_i, p_l)$ for some $p_l \in Piece^l_i(h) \cap T'_i$.

2. $p'_i = p_i$. We consider the ray $\overrightarrow{p'_i q}$. Suppose that the edge v_i hit by $\overrightarrow{p'_i q}$ separates $Cell(p'_i, T_j)$ and $Cell(p_o, T_j)$ for some point $p_o \in Piece^{l'}_i(h)$. Let q_e denote the first point where $\overrightarrow{p'_i q}$ intersects a cell $Cell(p'_i, q)$ for some $p'_i \in T'_i$. Every point r on $\overline{p'_i p_e}$ in $Cells(T_j, Piece^{l'}_i(h))$. Hence r is on $Cells(T'_i, Piece^{l'}_i(h) \cap T'_i)$. Since p_e is in $Cell(T'_i, p_e)$, the ray $\overrightarrow{p'_i q}$ intersects an edge v'_i at some point r, such that r is on $\overline{p'_i q_e}$. Therefore the edge v'_i separates $Cell(T'_i, p'_i)$ and $Cell(T'_i, p_l)$ for some $p_{l'} \in Piece^{l'}_i(h) \cap T'_i$.

When the index l (resp. l') is known, we can find the edge v'_i intersected by $\overrightarrow{p'_i q}$ in $O(\log\log n)$ time by binary search. □

Jump Procedure. We modify Step 1 of the Jump procedure. Suppose that we know the nearest neighbor p_i of q in T_i and we know that $dist(NN(T_i, q), q) \geq dist(NN_{[i, \frac{i+j}{2}]}(q), q)$ for some $j > i$. Using Lemma 11, we find $p'_i = NN(T'_i, q)$ in $O(\log\log n)$ time. We also find the edge $e'_i = \overline{v_1 v_2}$ of $Cell(T'_i, p'_i)$ that is intersected by $\overrightarrow{p_i q}$. Using the hull data structure for the triangle $\triangle(p'_i v_1 v_2)$, we look for the smallest j', such that $\frac{i+j}{2} \leq j' \leq j$, and q is outside of $Hull_i(j', p_i)$. If j' is found, we execute Steps 2-4 of the Jump procedure as described in Section 2.2. If there is no such hull, the procedure returns failure. The total runtime of the Jump procedure is $O(\log\log n + \log d(j-i)) = O((j-i)\log\log n)$.

Using the same analysis as in Section 2.1, the total runtime of the nearest neighbor procedure is $O(\log n)$: the runtime of the Jump procedure can be bounded by $\frac{c}{2}\log\log n(j-i)$ for some constant c; if we re-define the potential after the t-th jump to $\Phi_t \coloneqq c\log\log n(2i_t + j_t)$, then, using the same method as in Lemma 4, we can bound the runtimes of all jump procedures by $\Phi_T + O(T\log\log n)$ where T is the total number of jumps needed to answer a nearest neighbor query. Since $T \leq f = O(\log_d n)$, the overall cost of all jumps $O(\frac{\log n}{\log d}\log\log n) = O(\log n)$.

Piece Interior Search Structure. In order to complete the Jump procedure (see Step 3 in the case when the index j' is found), we need to find the Voronoi cell containing q provided that $Piece^l_i(k)$ is known. This step must be completed in $O(k\log\log n)$ time. To this aim, we need an additional data structure, described in the following lemma.

Lemma 13. *Suppose we know the piece $Piece^l_i(k)$ that contains the query point q. Then we can find the cell $Cell(T_i, p)$ containing q in time $O(k \cdot \log d)$. The underlying data structure uses space $O(m)$ and can be constructed in $O(m\log\log n)$ time, where m is the number of points in T_i.*

Proof. The set T_i is recursively divided into pieces $Pieces_i(k)$ where k is a power of 2 such that $h < k \leq (\log m)/4$. For every such k and for each $Piece_i^l(k)$, we consider all $Piece_i^{l'}(\frac{k}{2})$ such that $Piece_i^{l'}(\frac{k}{2}) \subset Piece_i^l(k)$. Let $PSep_i^l(k)$ denote the union of all $Sep_i^{l'}(k/2)$ such that $Piece_i^{l'}(k/2) \subset Piece_i^l(k)$. For every $Piece_i^l(k)$, we construct the Voronoi diagram of $PSep_i^l(k)$ and a data structure that answers point location queries. The total number of points in all $PSep_i^l(k)$ for all k and all l is $O(\frac{m}{\sqrt{g}})$: For $k = 2^j$, the number of points in all $PSep_i^l(k)$ is $O(\frac{m}{\sqrt{k}}) = O(\frac{m}{d^{2^{j-1}}})$. Summing over all k, such that $\lceil \log h \rceil < k < \log\log_d m$, the total number of points in all $PSep_i^l(k)$ is $O(\frac{m}{2^{\log h - 1}}) = O(\frac{m}{\sqrt{g}})$. Hence we can construct $Vor(PSep_i^l(k))$ for all $PSep_i^l(k)$ in $o(m)$ time. Additionally, for every $Piece_i^l(h)$, we also store its Voronoi diagram $Vor(Piece_i^l(h))$. All $Vor(Piece_i^l(h))$ use $O(m)$ space and can be constructed in $O(m \log g) = O(m \log\log n)$ time.

A query can be answered as follows. Suppose that q is known to be in $Piece_i^{l'}(k)$ for some $k > h$. We note that k is a power of 2. We set $k_0 = k$, $j = 0$ and repeat the following loop: We find $Cell(PSep_j^l(k_j), p_{k_j})$ that contains q and test whether q is in $Cell(T_i, p_{k_j})$. If this is the case, we stop. Otherwise we find the edge e' that is hit by the ray $\overrightarrow{p_{k_j} q}$. hen we increment j and set $k_j = k_{j-1}/2$. When e' is known, we can identify $Piece_i^{l_j}(k_j)$ that contains the query point q and start the next iteration of the loop. If $k_j = h$, we stop the loop and find $Cell(Piece_i^{l_j}(h), p)$ that contains q. The j-th iteration of the loop takes $O(\log(d^{k_j})) = O(k_j \log d)$ time. Hence a query is answered in $O(k_0 \log d) = O(k \log\log n)$ time. □

Insertions. Now we can evaluate the overall cost of constructing the level i of our data structure. The set T_i is updated by selecting a subset $\overline{T} \subset T_{i-1}$, such that $\overline{T}$ contains $d^{i-1}/2$ points, and moving points $p \in \overline{T}$ from T_{i-1} to T_i. We can choose the points of $\overline{T}$; for example, we can select $d^{i-1}/2$ points from T_{i-1} with the smallest x-coordinates. Hence we can obtain the Voronoi diagram of $\overline{T}$ in $O(|T_i|)$ time. We can also merge the Voronoi diagrams of $\overline{T}$ and T_i and obtain the Voronoi diagram of the updated set T_i in $O(|T_i|)$ time using a linear-time algorithm for merging Voronoi diagrams [7, 17]. When $Vor(T_i)$ is available, we obtain $Sample_i(k)$ for all $k \leq (1/4)\log_d n$ such that k is a power of 2. Each $Sample_i(k)$ can be obtained in $O(|T_i|)$ time. Hence we need $O(|T_i| \log\log n)$ time to obtain all $Sample_i(k)$ for $k \leq (1/4)\log_d n$ such that k is a power of 2. Then we re-build Voronoi diagrams for T_j, $j < i$: for every $j < i$, we construct the Voronoi diagram of $T_j^{\mathrm{aux}} = \cup_{j'>j} Sample_{j'}(\mathrm{pow2}(\max(j' - j, h)))$. Since this set contains $O(|T_j|/g)$ points, we can construct the Voronoi diagram of T_j^{aux} in $O(|T_j|)$ time. Finally we merge $Vor(T_j^{\mathrm{aux}})$ and the Voronoi diagram of S_j and obtain $Vor(T_j)$ in $O(|T_j|)$ time.

When $Vor(T_i)$ is available, we can extract the points of T_i', construct $Vor(T_i')$ and all convex hull structures in $O(|T_i|)$ time. We can also construct the piece interior search structure for T_i in $O(|T_i| \log\log n)$ time. For each $j < i$, we can construct T_j' and auxiliary data structures in $O(|T_j| \log\log n)$ time. In summary, we can re-build the level i and levels $j < i$ of our data structure in time $O(|T_i| \log\log n)$.

Using the standard logarithmic method analysis, the overall insertion time is $O(\log n \cdot d \cdot \log\log n) = O(\log^{1+\varepsilon} n \log\log n)$ time. If we replace ε with $\varepsilon' < \varepsilon$ in the above description, the insertion time is reduced to $O(\log^{1+\varepsilon'} n \log\log n) = O(\log^{1+\varepsilon} n)$. We obtain the following result.

Theorem 2. *There exists a semi-dynamic insertion-only data structure that answers two-dimensional nearest neighbor queries in $O(\log n)$ time and supports insertions in $O(\log^{1+\varepsilon} n)$ amortized time. The data structure uses $O(n)$ space.*

Appendix C. Offline Persistent Data Structure

In this section we explain how our method can be used in the offline partially persistent fully dynamic scenario, i.e., in the case when the entire sequence of updates is known in advance and we can ask nearest neighbor queries to any version of the data structure.

We associate a *lifespan* with every point p. If p was inserted at time t_1 and removed at time t_2, then the lifespan of p is $[t_1, t_2]$; we assume $t_2 = \infty$ if p is never removed. All points are stored in a variant of the segment tree data structure. The leaves of the segment tree $\mathcal{T}$ store the insertion and deletion times of different points in sorted order. Each internal node has $\log^{\varepsilon} n$ children. The range $rng(u)$ of a node u is the interval $[u_{\min}, u_{\max}]$ where $u_{\min}$ ($u_{\max}$) is the value stored in the leftmost (rightmost) leaf descendant of u. We store a point p in the set $S(u)$ if $rng(u) \subseteq [t_1(p), t_2(p)]$, but $rng(\mathrm{parent}(u)) \not\subseteq [t_1(p), t_2(p)]$ where $[t_1(p), t_2(p)]$ is the lifespan of the point p. Each point is stored in $O(\log^{1+\varepsilon} n)$ sets $S(u)$. If the lifespan of a point p includes time t, then there is exactly one node u such that $p \in S(u)$ and u is an

ancestor of the leaf ℓ_t that holds the value t. In order to find the nearest neighbor of a point q at time t we must find the nearest neighbor of q in $\cup_{u\in\pi(t)}S(u)$ where the union is taken over all nodes on the path $\pi(t)$ from the root to ℓ_t.

Following the approach of Sections 2 and B, we define $T(u) \supseteq S(u)$ for all nodes of $\mathcal{T}$. For the root node u_R, $T(u_R) = S(u_R)$. For a non-root node u, we denote by $\mathrm{anc}_j(u)$ the height-j ancestor of u and let $T_j(u) = T(\mathrm{anc}_j(u))$. Sets $Sample_j(k, u)$ are defined with respect to $T_j(u)$ in the same way as in Section 2.1. We set $T(u) = S(u) \cup_{j>\mathrm{height}(u)} Sample_j(j-i, u)$. Thus sets $T(u)$ can be constructed for all nodes $u \in \mathcal{T}$ in top-to-bottom order. When $T(u)$ is defined, we can construct $T'(u)$ and all auxiliary data structures necessary for the Jump procedure. Now we can find $NN(\cup_{u\in\pi(t)}S(u), q) = NN(\cup_{u\in\pi(t)}T(u), q)$ as follows. We start in a leaf node of $\pi(t)$ and move up along $\pi(t)$ using the Jump procedure. Using the analysis of Lemma 4, the total cost of all Jumps is $O(\log n)$.

Theorem 3. *There exists an offline partially persistent data structure that answers two-dimensional nearest neighbor queries in $O(\log n)$ time. The data structure uses $O(n \log^{1+\varepsilon} n)$ space and can be constructed in $O(n \log^{1+\varepsilon} n)$ time.*

Appendix D. Semi-Online Fully-Dynamic Data Structure

In this section we consider the ephemeral (non-persistent) offline scenario. In this scenario the sequence of all insertions and deletions is known in advance. However only the latest version of the data structure can be queried.

We order updates and assign every update an integer timestamp. As in Section C we associate a lifespan $[t_1(p), t_2(p)]$ with every point p; we also maintain a tree $\mathcal{T}$, so that the leaves of $\mathcal{T}$ store the insertion and deletion times of different points in sorted order. The definition of a set $S(u)$ is changed so that every point is stored in only one set at every time. Let `cur` denote the timestamp of the current version. The set $S(u)$ contains all points p such that $rng(u) \subseteq [t_1(p), t_2(p)]$, $rng(\mathrm{parent}(u)) \not\subseteq [t_1(p), t_2(p)]$, and $u_{\min} \le \mathtt{cur} \le u_{\max}$.

After every update, we modify our data structure as follows. We examine sets $S(u)$ for every node u, such that $u_{\max} = \mathtt{cur}$. For every such node u and for each $p \in S(u)$, we remove p from $S(u)$ and insert it into $S(u')$ such that $rng(u') \subseteq [t_1(p), t_2(p)]$, $rng(\mathrm{parent}(u')) \not\subseteq [t_1(p), t_2(p)]$, and $u'_{\min} \le \mathtt{cur}+1 \le u'_{\max}$. If such u' does not exist, we remove p from our data structure. When all sets $S(u)$ are processed, we increment `cur` by 1. Then we examine all nodes v, such that $v_{\min} = \mathtt{cur}$ in the decreasing order of their height. For each v we construct $T(v) = S(v) \cup (\cup_{j>\mathrm{height}(u)} Sample_j(j-i, u))$. We also construct and store all auxiliary data structures.

Every point p is inserted into $O(\log^{1+\varepsilon} n)$ different sets because there are $O(\log n)$ nodes u, such that $rng(u) \subseteq [t_1(p), t_2(p)]$ but $rng(\mathrm{parent}(u)) \not\subseteq [t_1(p), t_2(p)]$. Every set $S(u)$ is created only once. Hence the amortized update time is $O(\frac{c(n)}{n} \log^{1+\varepsilon} n) = O(\log^{1+2\varepsilon} n)$, where $c(n)$ is the construction time of the data structure. By replacing ε with $\varepsilon/2$, we obtain the following result.

Theorem 4. *There exists a fully-dynamic offline data structure that answers two-dimensional nearest neighbor queries in $O(\log n)$ time. The data structure uses $O(n)$ space where n is the total number of updates. Each update operation is supported in $O(\log^{1+\varepsilon} n)$ amortized time.*

The data structure of Theorem 4 can be adjusted to the semi-online scenario. In this setting the sequence of updates is not known in advance. But we know the deletion time $t_2(p)$ of each point p at the time when p is inserted. We initially create the tree $\mathcal{T}$ with $n' = 2n_0$ leaves where n_0 is the number of points initially stored in the data structure. When a new point with lifespan $[t_1(p), t_2(p)]$ is inserted, we set $t'_1(p) = t_1(p)$ and $t'_2(p) = \min(t_2(p), \mathtt{init} + n' - 1)$. The variable `init` is set to 1 when the data structure is initialized and updated every time when we re-build the tree $\mathcal{T}$. We insert p with lifespan $[t'_1(p), t'_2(p)]$ into $\mathcal{T}$ as described in the proof of Theorem 4.

After $n_0/2$ insertions or deletions, we set n_0 to the current number of elements in the data structure and build a new tree with $n' = 2n_0$ leaves. All points currently stored in the nodes of the old tree are moved to the new tree. For each point p, we set $t'_1(p) = \mathtt{cur}$ and $t'_2(p) = \min(t_2(p), \mathtt{cur} + n' - 1)$. For simplicity, all leaves in the new tree are indexed with `cur`, `cur` + 1, …, `cur` + n' − 1. Finally, we set `init` = `cur` and discard the old tree. The global re-building incurs $O(1)$ insertions into the new tree per update. Hence, the amortized update cost is unchanged.

Corollary 2. *There exists a fully-dynamic semi-online data structure that answers two-dimensional nearest neighbor queries in $O(\log n)$ time. The data structure uses $O(n)$ space where n is the total number of updates. Each update operation is supported in $O(\log^{1+\varepsilon} n)$ amortized time.*

P	A generic set of points with no specific meaning
S	The set of points currently stored
n	$\|S\|$
$dist(p, q)$	Distance from point p to q
$\triangle pqr$	Triangle with endpoints p, q, and r
d	A parameter initially set to a constant
$\mathcal{S} = \{S_1, S_2, \ldots S_f\}$	A partition of S, where $\|S_i\| = \Theta(d^i)$
f	$\|\mathcal{S}\|$
T_i	$S_i \cup \bigcup_{j=i+1}^{\|\mathcal{S}\|} Sample_j(j-i)$
$Vor(P)$	The Voronoi diagram of P
$G(P)$	The Delaunay graph of P
$Cell(P, p)$	The cell of p in $Vor(P)$
$Cells(P, P')$	For $P' \subseteq P$, the union of $Cell(P, p)$ for all $p \in P'$.
$NN(P, q)$	The nearest neighbor of q in P.
$NN_R(q)$	$\arg\min_{p \in \{T_k \mid k \in R \text{ and } k \in \mathbb{Z}\}} dist(p, q)$
$Pieces_j(k) = \{Piece_j^1(k), \ldots, Piece_j^{\|Pieces_j(k)\|}(k)\}$	A division of T_j into $\Theta(\|T_j\|/d^{4k})$ subsets such that each subset has size $O(d^{4k})$.
$Sep_j^\ell(k)$	The subset of points in $Piece_j^\ell(k)$ who have neighbors in $G(T_j)$ that are not in $Piece_j^\ell(k)$.
$\overline{Sep}_j^\ell(k)$	$Piece_j^\ell(k) \setminus Sep_j^\ell(k)$
$Seps_j(k)$	$\{Sep_j^1(k), Sep_j^2(k), \ldots Sep_j^{\|Pieces_j(k)\|}(k)\}$
$Sample_j(k)$	$\bigcup_{Sep \in Seps_j(k)} Sep$
$Hull_i(j, p_i)$, $p_i \in T_i$	The convex hull of $\{p_i\} \cup (Cell(T_i, p_i) \cap Cells(T_j, Sep_j(j-i)))$
$\overline{Hull}_i(j, p_i)$, $p_i \in T_i$	$Cell(T_i, p_i) \setminus Hull_i(j, p_i)$

TABLE 2. Table of notation.

REFERENCES

[1] Pankaj K. Agarwal and Jirí Matousek. Dynamic half-space range reporting and its applications. *Algorithmica*, 13(4):325–345, 1995. doi:10.1007/BF01293483.

[2] Sarah R. Allen, Luis Barba, John Iacono, and Stefan Langerman. Incremental Voronoi diagrams. *Discrete and Computational Geometry*, 58(4):822–848, 2017. doi:10.1007/s00454-017-9943-2.

[3] Jon Louis Bentley and James B. Saxe. Decomposable searching problems I: static-to-dynamic transformation. *Journal of Algorithms*, 1(4):301–358, 1980. doi:10.1016/0196-6774(80)90015-2.

[4] Timothy M. Chan. A dynamic data structure for 3-d convex hulls and 2-d nearest neighbor queries. *Journal of the ACM*, 57(3):16:1–16:15, 2010. doi:10.1145/1706591.1706596.

[5] Timothy M. Chan. Dynamic geometric data structures via shallow cuttings. *Discrete and Computational Geometry*, 64(4):1235–1252, 2020. doi:10.1007/s00454-020-00229-5.

[6] Timothy M. Chan and Konstantinos Tsakalidis. Optimal deterministic algorithms for 2-d and 3-d shallow cuttings. *Discrete and Computational Geometry*, 56(4):866–881, 2016. doi:10.1007/s00454-016-9784-4.

[7] Bernard Chazelle. An optimal algorithm for intersecting three-dimensional convex polyhedra. *SIAM Journal on Computing*, 21(4):671–696, 1992. doi:10.1137/0221041.

[8] Bernard Chazelle and Leonidas J. Guibas. Fractional cascading: I. A data structuring technique. *Algorithmica*, 1(2):133–162, 1986. doi:10.1007/BF01840440.

[9] Bernard Chazelle and Ding Liu. Lower bounds for intersection searching and fractional cascading in higher dimension. *Journal of Computing & Systems Sciences*, 68(2):269–284, 2004. doi:10.1016/j.jcss.2003.07.003.

[10] Kenneth L. Clarkson, Kurt Mehlhorn, and Raimund Seidel. Four results on randomized incremental constructions. *Computational Geometry*, 3:185–212, 1993. doi:10.1016/0925-7721(93)90009-U.

[11] Mark de Berg, Otfried Cheong, Marc J. van Kreveld, and Mark H. Overmars. *Computational geometry: algorithms and applications, 3rd Edition*. Springer, 2008.

[12] Olivier Devillers, Stefan Meiser, and Monique Teillaud. Fully dynamic Delaunay triangulation in logarithmic expected time per operation. *Computational Geometry*, 2:55–80, 1992. doi:10.1016/0925-7721(92)90025-N.

[13] David P. Dobkin and Subhash Suri. Maintenance of geometric extrema. *Journal of the ACM*, 38(2):275–298, 1991. doi:10.1145/103516.103518.

[14] Greg N. Frederickson. Fast algorithms for shortest paths in planar graphs, with applications. *SIAM Journal on Computing*, 16(6):1004–1022, 1987. doi:10.1137/0216064.

[15] Michael T. Goodrich. Planar separators and parallel polygon triangulation. *Journal of Computer and Systems Sciences*, 51(3):374–389, 1995. doi:10.1006/jcss.1995.1076.

[16] Haim Kaplan, Wolfgang Mulzer, Liam Roditty, Paul Seiferth, and Micha Sharir. Dynamic planar Voronoi diagrams for general distance functions and their algorithmic applications. *Discrete and Computational Geometry*, 64(3):838–904, 2020. doi:10.1007/s00454-020-00243-7.

[17] David G. Kirkpatrick. Efficient computation of continuous skeletons. In *20th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 18–27, 1979. doi:10.1109/SFCS.1979.15.

[18] David G. Kirkpatrick. Optimal search in planar subdivisions. *SIAM Journal on Computing*, 12(1):28–35, 1983. doi:10.1137/0212002.

[19] Philip N. Klein, Shay Mozes, and Christian Sommer. Structured recursive separator decompositions for planar graphs in linear time. In *Symposium on Theory of Computing (STOC)*, pages 505–514, 2013. doi:10.1145/2488608.2488672.

[20] Joseph S.B. Mitchell and Wolfgang Mulzer. Proximity algorithms. In Jacob E. Goodman, Joseph O'Rourke, and Csaba D. Tóth, editors, *Handbook of Discrete and Computational Geometry*, chapter 32, pages 849–874. CRC Press, 2017.

[21] Ketan Mulmuley. *Computational geometry — an introduction through randomized algorithms*. Prentice Hall, 1994.

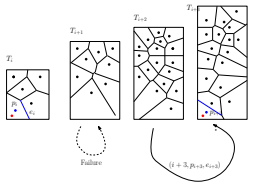


FIGURE 3. Two iterations of the Jump procedure. The query point q is shown in red. Points $p_i = NN(T_i, q)$, $p_{i+3} = NN(T_{i+3}, q)$, and edges e_i and e_{i+3} are shown in blue.

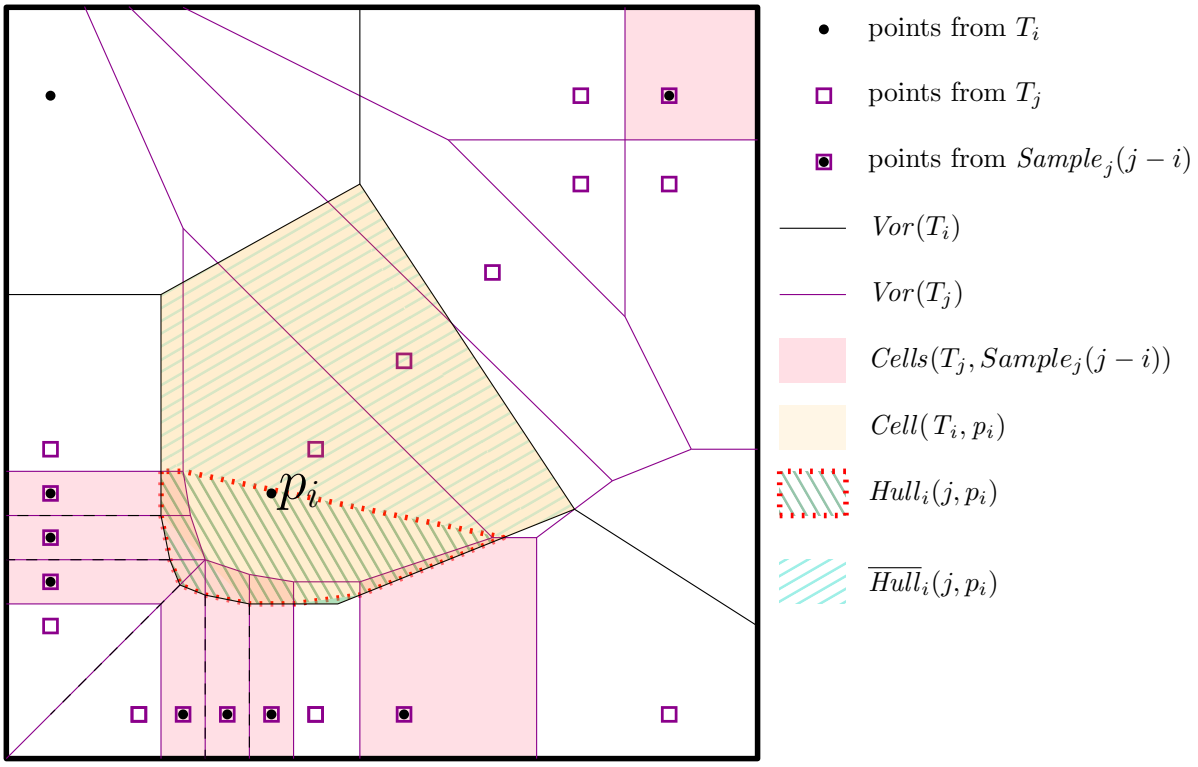


FIGURE 4. Illustration of the computation of $Hull_i(j, p_i)$ and $\overline{Hull}_i(j, p_i)$. Observe that $Hull_i(j, p_i)$ is the convex hull of those parts of $Cells(T_j, Sample_j(j-i))$ (shaded pink) that are inside $Cell(T_i, p_i)$ (shaded tan). $\overline{Hull}_i(j, p_i)$ is simply the remainder of $Cell(T_i, p_i)$, and has two connected components, including a small one at the bottom. By Lemma 7, the closest point in $T_i \cup T_j$ to all points in $Hull_i(j, p_i)$ is p_i.